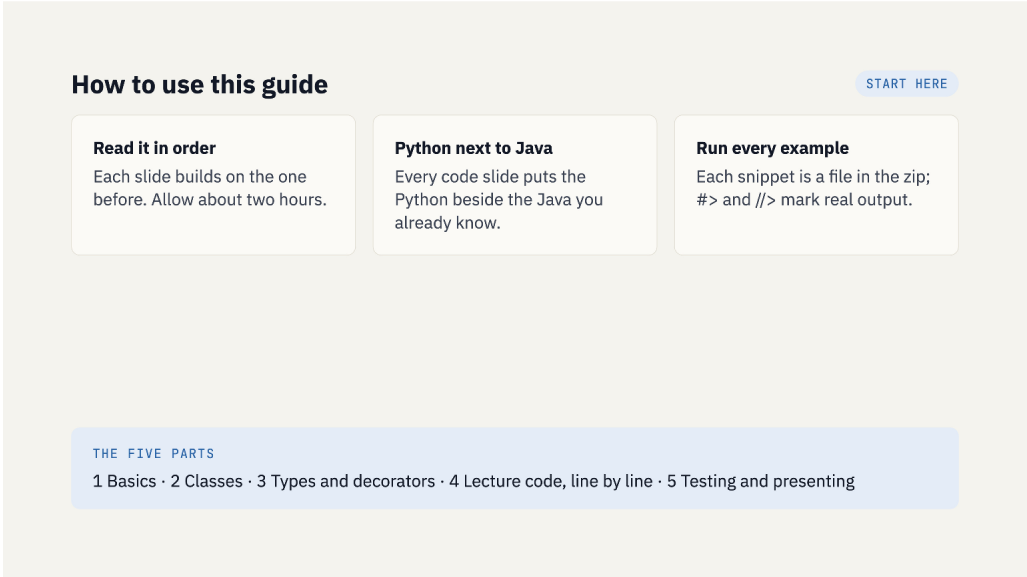
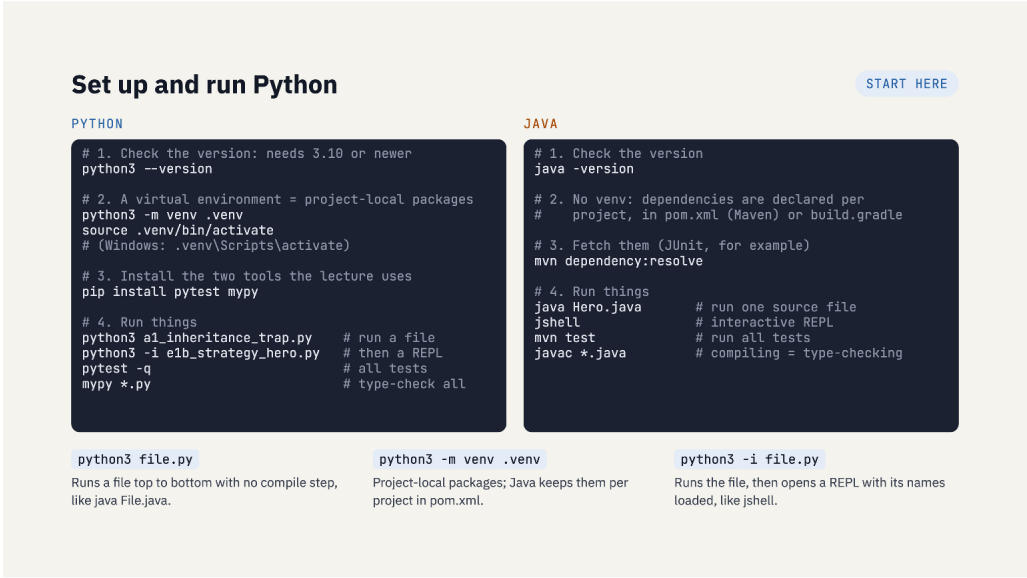


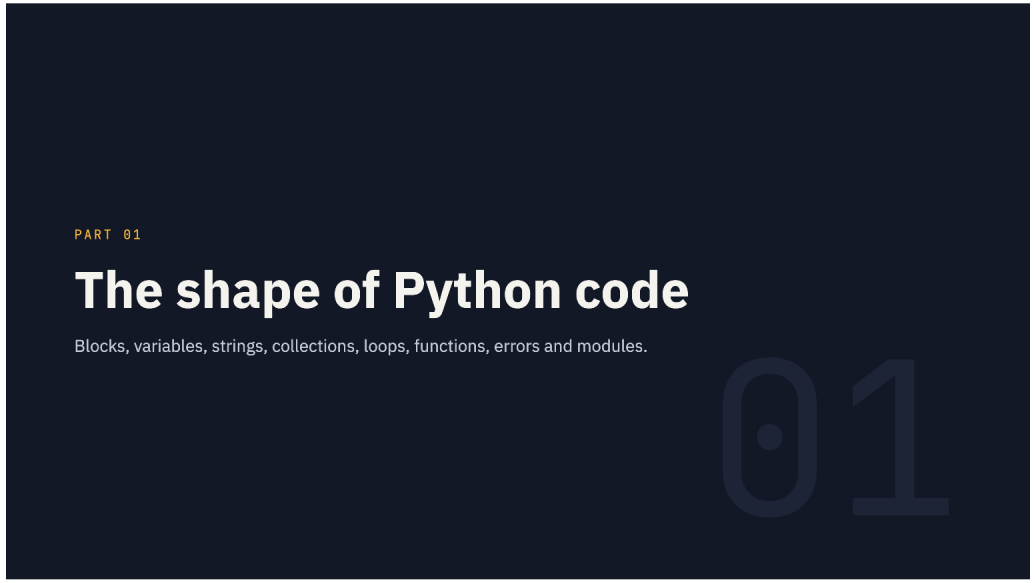
This guide is for you, the presenter, not the audience. It walks through every piece of Python that appears in the lecture deck, and every code slide puts the same example in Java next to it, so you can map each line onto what you already know. Work through it once with the companion files open and run each example.



Python is always on the left, Java on the right. The Java isn’t there to be learned; it’s the anchor, so you can see what each Python line corresponds to. Lines ending in #> show what Python printed, //> what Java printed; #! is mypy’s output and //! the compiler’s. All of them were produced by running the code.



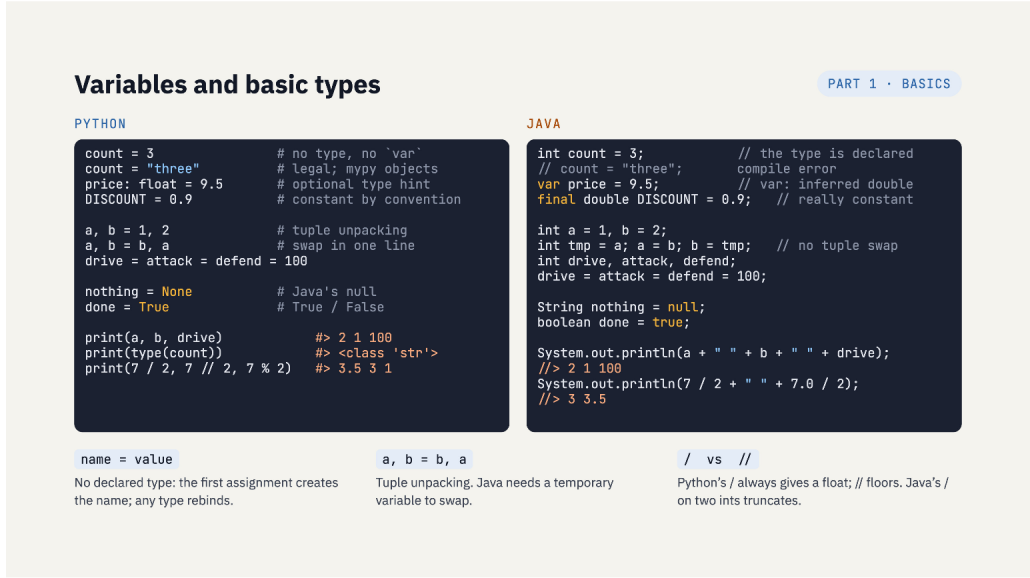
Do this once on the laptop you'll present from. Unzip the companion folder, create the venv inside it, install pytest and mypy, and run pytest -q: you should see 27 passed. Activate the venv in every new terminal. python3 -i is the best presenting trick: it runs a file and leaves you at a >>> prompt with everything loaded.



Part 1: the basics you need to read any line of the lecture code.



Indentation is syntax, not style: the body of the if is whatever is indented under it. Mixing tabs and spaces is an error, so let the editor insert 4 spaces. A statement ends at the end of the line; to continue a long expression, keep it inside (), [] or {}. There's no new keyword. On the Java side of every slide, statements shown without a method are the body of main().



Think of every Python variable as a Java reference without a declared type. Integers never overflow. Constants are a naming convention only. The chained assignment drive = attack = defend = 100 appears in the lecture's Decorator example, and it works in Java too.



Strings are immutable in both languages. One trap if you’re in Germany: Java’s %.2f uses the default locale, so on a German machine it prints 15,50. Python’s format spec never uses the locale. removesuffix, used in the Command example, needs Python 3.9 or newer. Triple-quoted strings are Python’s text blocks.



These literals cover almost every data structure in the lecture: the observer list, the factory's dict registry, and tuples returned by functions. Note that len(x) is a function, not a method, and [-1] is the last element (Java 21: getLast()). {} on its own is an empty dict; an empty set is set().



The lecture’s OCP ‘before’ example is an if/elif chain, and nearly every loop in the lecture is a for-each. Python has while, break and continue, but no do-while and no ++: write i += 1. Python 3.10 added match, similar to Java’s switch.



This matters because the lecture uses power or RegularJoe() and confusing or Confusing(). It reads well, but be ready for the trap: count or 10 gives 10 when count is 0. When 0 or "" are valid values, test is None.



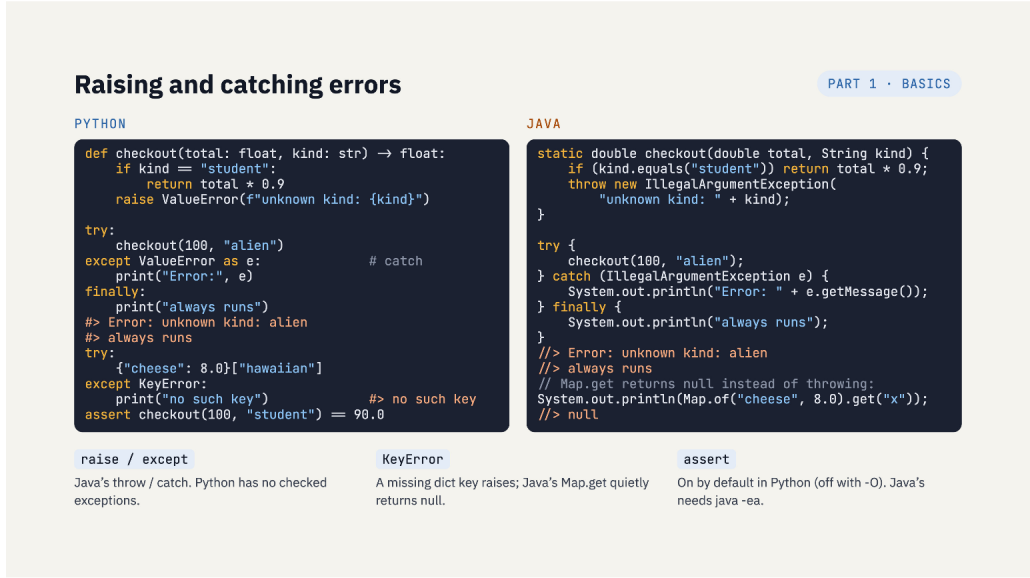
Functions are defined with def and can live at module level; they don’t need a class. There is no overloading: a second def with the same name replaces the first. A function without return gives back None, like a void method. Note area(3, 2) printed 6, an int: hints don’t convert anything.



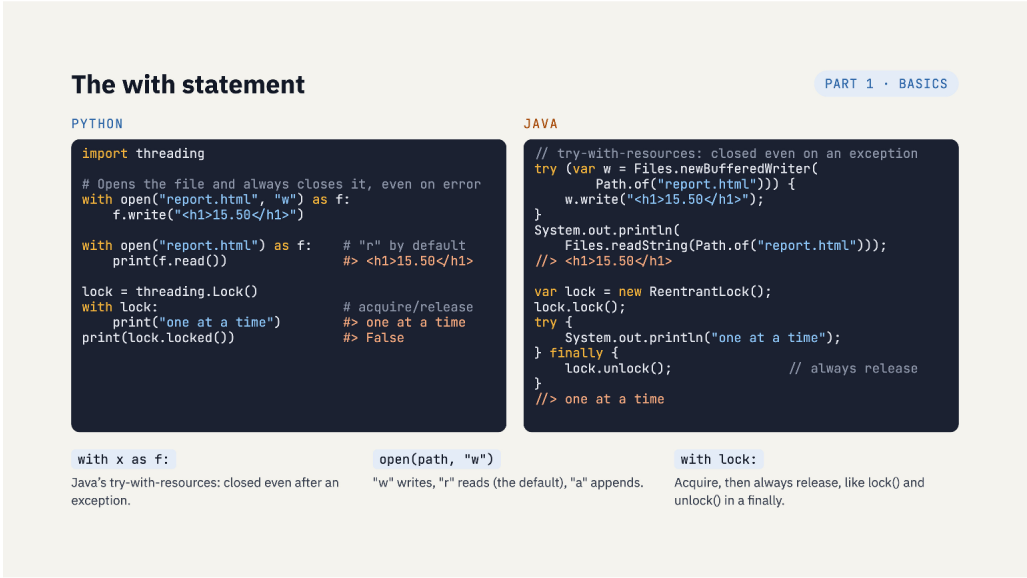
This slide unlocks the Facade’s add_then_multiply(self, *nums, factor) and its call calls[len(nums)](*nums, factor). A Python lambda is limited to one expression; Java lambdas can have blocks. **kwargs collects extra keyword arguments into a dict; the decorators slide uses it.



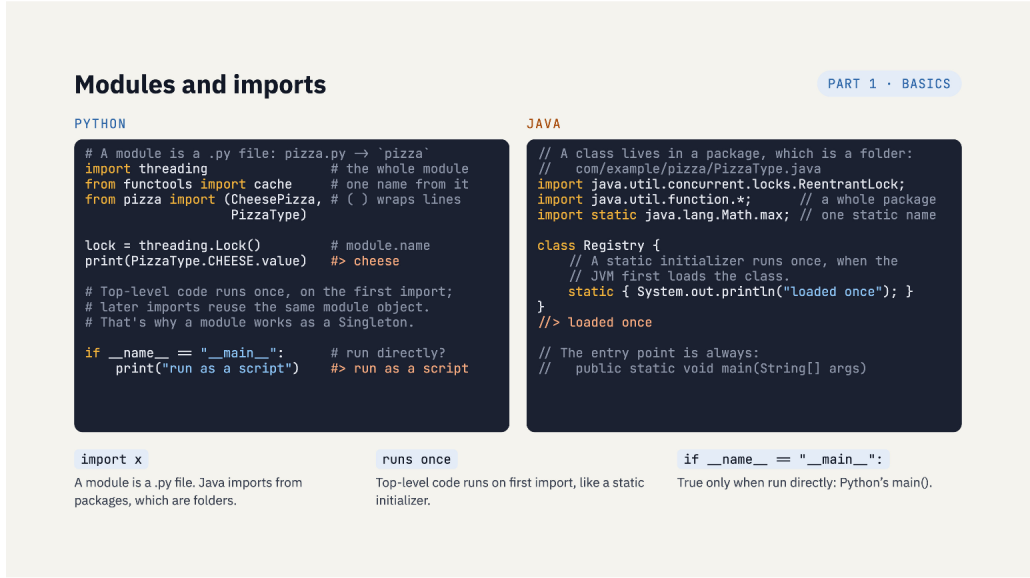
The lecture uses generator expressions in the SRP slide (sum of prices), in the composition example (join of every power) and in the Liskov example. Python dicts keep insertion order; a Java HashMap promises no order, even if it happens to print in order here.



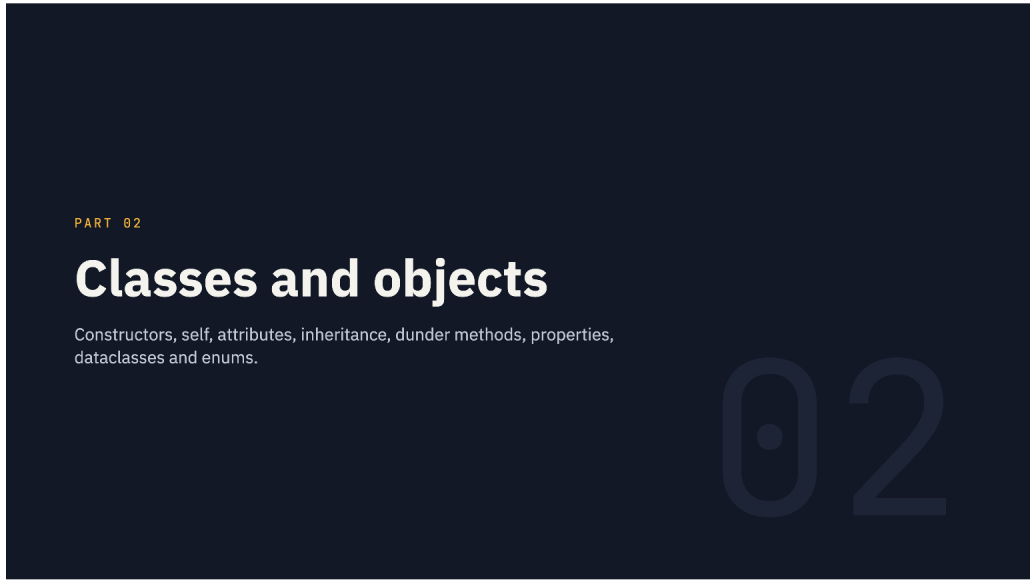
The lecture raises ValueError in the OCP example, Exception in the repo's BadGuyAdapter and NotImplementedError in the ISP example. Common ones: KeyError for a missing key, AttributeError for a missing attribute or method, TypeError for wrong arguments, including instantiating an abstract class.



Any object with __enter__ and __exit__ works in a with statement, like AutoCloseable in Java. The SRP example saves a file this way and the Singleton uses with cls._lock. pytest.raises is a with block too.



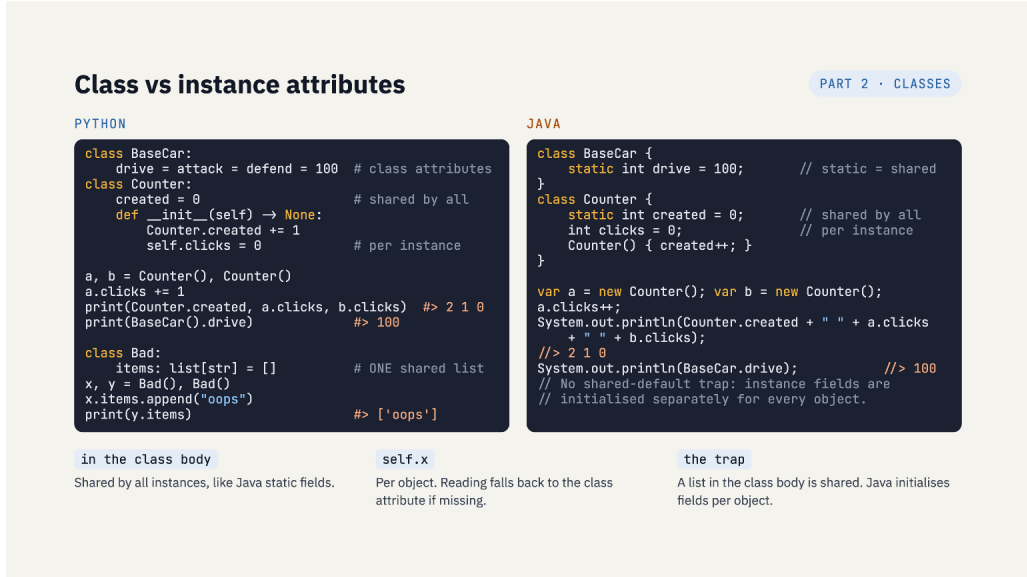
Because a module runs only once and is then cached, a module-level object is effectively a singleton, which is the Pythonic Singleton in the lecture, just as Java's class loading makes the holder idiom safe.



Part 2: classes. Almost every lecture example is a handful of small classes, so this part matters most.



The two programs are equivalent. No new; the constructor is a method called __init__; self is explicit in every method signature but never passed by the caller; and the field is created by assigning it. The underscore in _power is Python's spelling of private, two slides on.



Name lookup goes object first, then class, then parent classes, which is why BaseCar().drive works even though BaseCar has no __init__. In the lecture, Singleton’s _instance and _lock, BaseCar’s stats and the factory’s _registry are class attributes.



If asked, the Python motto is ‘we’re all consenting adults’: the language trusts conventions. The commented Java line really fails to compile with ‘balance has private access’. Underscores in 1_000_000 work in both languages.



All methods are virtual in both languages. The one place Java developers trip up: in Java the parent constructor runs automatically (super() must come first); in Python only if you call it. isinstance is instanceof.



This is the lecture’s ‘inheritance trap’. The nearest Java gets is two interfaces with default methods, and javac rejects the class (‘inherits unrelated defaults’) until you override and pick. Python silently picks for you, which is the lecture’s point: behaviors don’t merge.



Double-underscore methods are hooks the language calls for you: len(), str(), ==, +, [], in and for loops. The lecture's composed Stack defines __len__, and the Singleton overrides __new__, covered in Part 4. @dataclass(frozen=True) writes __eq__ and __hash__ for you.



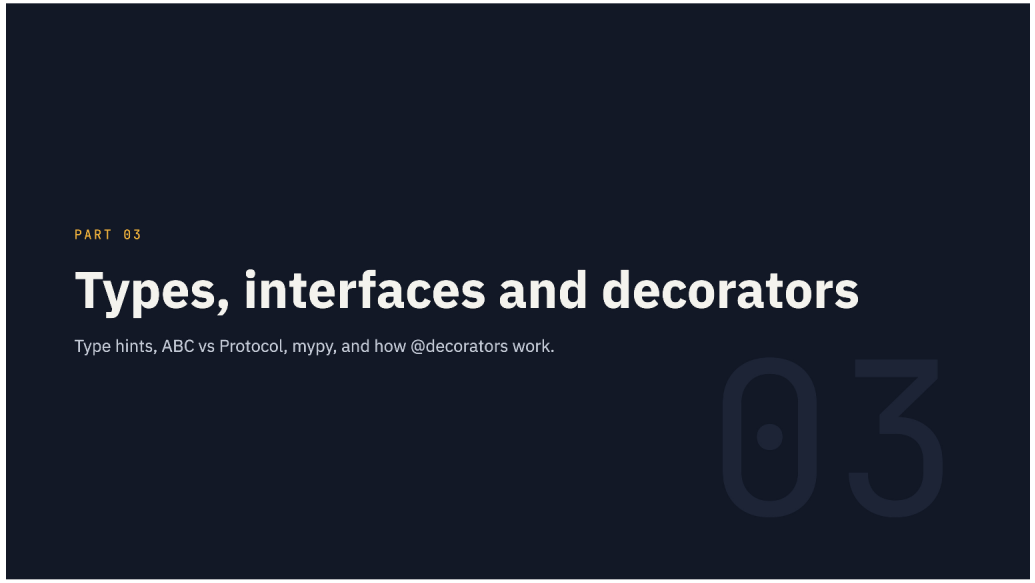
Without a setter, assigning car.drive = 5 raises AttributeError. Python style starts with plain attributes and switches to @property only when needed, which is why Python code has so few getters. Every Decorator-pattern car in the lecture uses properties.



Fields are declared with type hints; their order is the constructor's parameter order, and fields with defaults come last. Assigning r.score on a frozen dataclass raises FrozenInstanceError. A plain = [] default is refused, because every instance would share one list. GameResult and Pizza in the lecture are dataclasses.



PizzaType is the enum the factory examples use as dict keys. Members are singletons, so is and == both work, just as == is safe for Java enums.



Part 3: the parts of the lecture code that look least like Java.



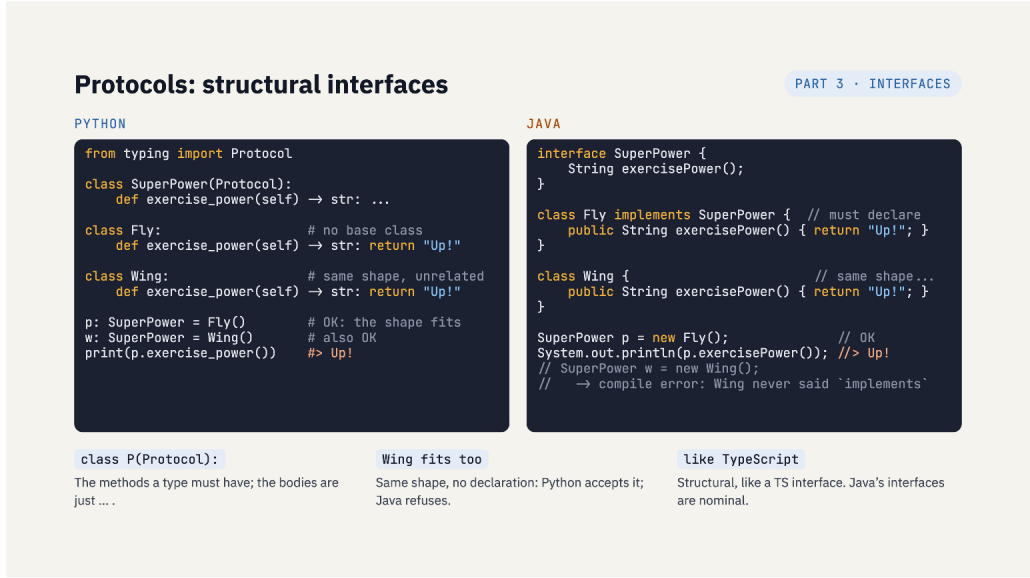
The most important fact for a Java developer: types are optional and not enforced by the interpreter. Treat mypy as the compiler's type checker that you run separately, ideally in CI. list[str] and dict[str, int] are built-in generics (3.9+); str | None needs 3.10. The #! and //! lines are the real tool output.



The Simple Factory's registry maps enum members to Callable[[], Pizza]: each value is a class, and a class is a callable that returns an instance. In Java the equivalent is a constructor reference, CheesePizza::new.



The lecture uses ABCs where the base class also carries code: Factory Method's order_pizza and Template Method's create_pizza_for_delivery. The body ... is a placeholder, like an empty abstract method.



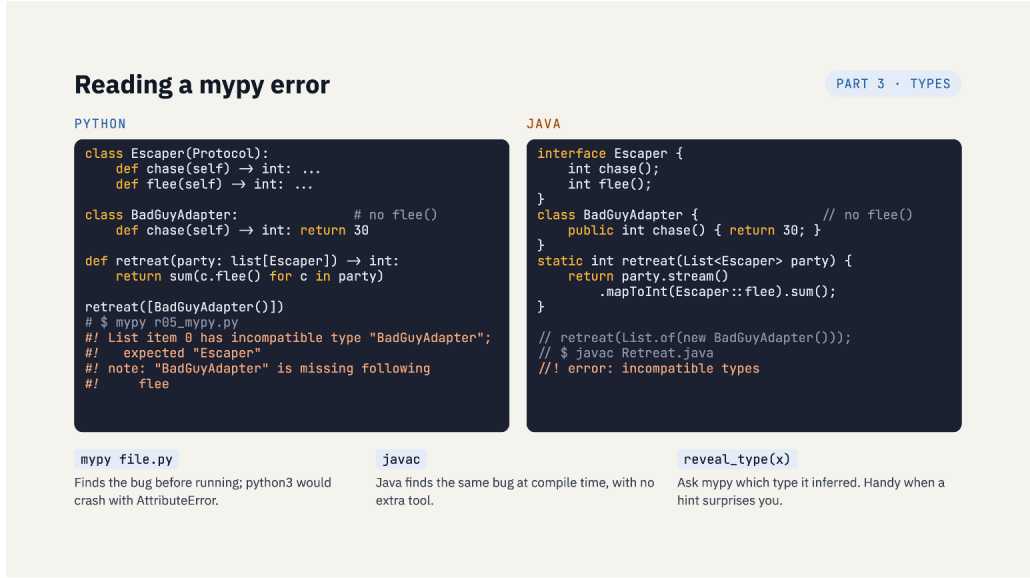
You know TypeScript, which makes this easy: a Protocol is structural typing, exactly like a TS interface. Wing never mentions SuperPower, yet mypy accepts it. The Java side shows the opposite: without implements, javac rejects it. Most interfaces in the lecture are Protocols.

Protocol or ABC?

PART 3 · INTERFACES

	Protocol	ABC
Classes must inherit it	no	yes
Checked	by mypy, before running	when you instantiate
Shares code with implementers	only if they subclass it	yes
Fits third-party classes	yes	no
Closest Java	none: interfaces need implements	abstract class
Used in the lecture for	most interfaces	Factory and Template Method

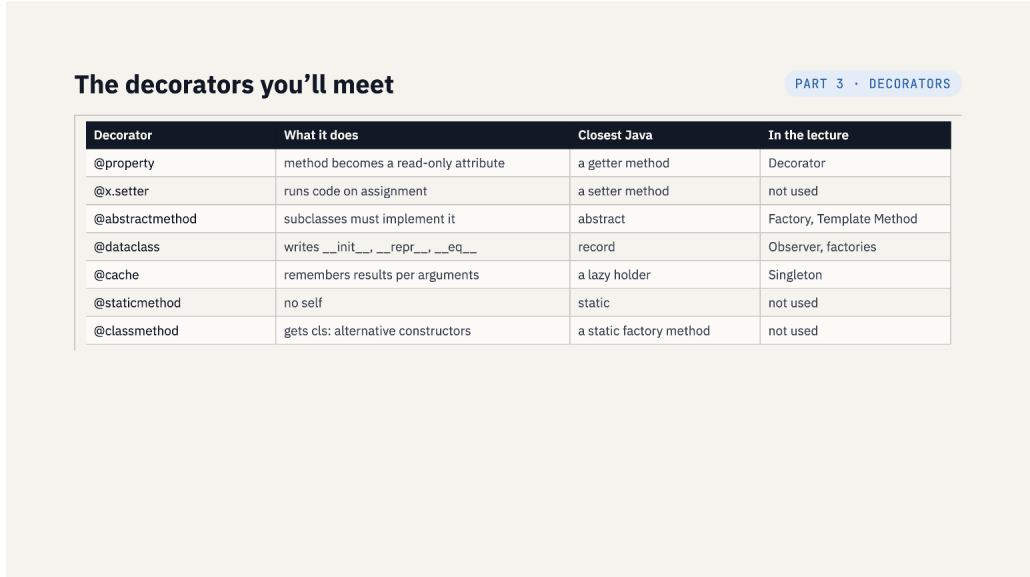
A short rule for the audience question: use a Protocol when you only need a shape, and an ABC when the base class carries shared code or you want instantiation to fail early. Java has no structural equivalent of Protocol.



This is the lecture’s Liskov fix in miniature. The error says: in list item 0 you passed a BadGuyAdapter where an Escaper was expected, and the note names the missing member, flee. mypy skips functions with no hints at all; mypy --strict demands hints everywhere.



A decorator is a function that takes a function and returns a replacement; with arguments, as in @dataclass(frozen=True), the call runs first and returns the real decorator. You don’t need to write decorators for the lecture, but @property, @abstractmethod, @dataclass and @cache are all this mechanism. Java’s closest equivalent is a higher-order function, as on the right.



The decorators you'll meet

PART 3 · DECORATORS

Decorator	What it does	Closest Java	In the lecture
@property	method becomes a read-only attribute	a getter method	Decorator
@x.setter	runs code on assignment	a setter method	not used
@abstractmethod	subclasses must implement it	abstract	Factory, Template Method
@dataclass	writes __init__, __repr__, __eq__	record	Observer, factories
@cache	remembers results per arguments	a lazy holder	Singleton
@staticmethod	no self	static	not used
@classmethod	gets cls: alternative constructors	a static factory method	not used

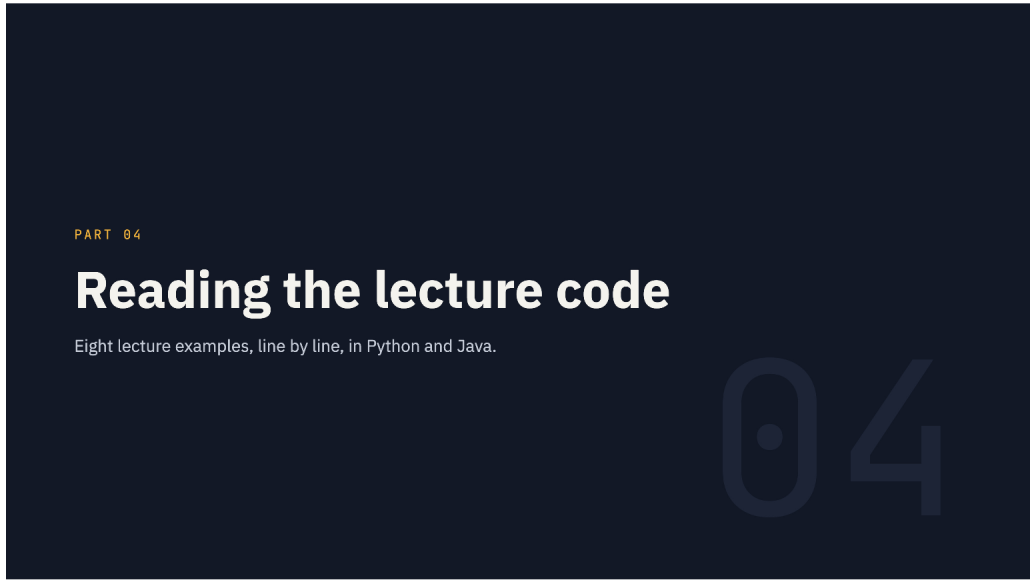
@staticmethod and @classmethod aren't in the lecture, but someone may ask. A classmethod receives the class as cls, which is how Python writes alternative constructors such as dict.fromkeys.

Standard-library tools in the code

PART 3

Tool	What it does	Closest Java
threading.Lock	one thread at a time, used with with	synchronized (not reentrant)
functools.cache	runs once per set of arguments	a lazy holder
functools.partial	pre-fills some arguments	a lambda capturing values
weakref.WeakSet	a set that doesn’t keep objects alive	WeakHashMap keys
decimal.Decimal	exact decimal arithmetic	BigDecimal
open(path, mode)	file I/O, closed by with	Files.newBufferedWriter

threading.Lock is not reentrant: the same thread acquiring it twice deadlocks. threading.RLock is the reentrant one, like Java’s ReentrantLock.



Part 4: eight of the lecture’s code slides, exactly as they appear there, with the syntax called out.



Reading order: the Hero context class, a subclass that picks a default, then three lines of usage. do_hero_stuff simply forwards to the held power. The comments show what each call returns.

Decorator, line by line

LECTURE CODE

PYTHON

```python
class BaseCar:
    drive = attack = defend = 100

class ArmoredCar:                  # wraps ANY car
    def __init__(self, car) -> None:
        self._car = car
    @property
    def drive(self) -> int:
        return self._car.drive - 20
    @property
    def attack(self) -> int:
        return self._car.attack
    @property
    def defend(self) -> int:
        return self._car.defend + 40

armored = ArmoredCar(BaseCar())
print(armored.drive, armored.defend)    # 80 140
```

JAVA

```java
interface Car {
    int drive(); int attack(); int defend();
}
record BaseCar() implements Car {
    public int drive()  { return 100; }
    public int attack() { return 100; }
    public int defend() { return 100; }
}
record ArmoredCar(Car car) implements Car { // wraps
    public int drive()  { return car.drive() - 20; }
    public int attack() { return car.attack(); }
    public int defend() { return car.defend() + 40; }
}

// in main():
Car armored = new ArmoredCar(new BaseCar());
// armored.drive(), armored.defend()  → 80, 140
```

`drive = attack = defend = 100`

Three class attributes in one line; Java's record returns 100.

`def __init__(self, car)`

No type on car: duck typing. Java demands the Car interface.

`@property`

armored.drive reads like a field; Java calls armored.drive().

Each wrapper has the same three properties, so a wrapped car is still a car. print with several arguments separates them with spaces, which is why the output is 80 140.



Settings() calls __new__ first and then __init__ on whatever __new__ returned. This class has no __init__, so the shared instance is never re-initialised. If someone adds one, it runs on every Settings() call, one more reason the lecture recommends the @cache or module version.



pizza.py in the companion folder defines PizzaType as an Enum and the pizzas as dataclasses. Parentheses let the import span two lines. This is the slide where ‘classes are callables’ really matters.



The copy matters: changing a list while looping over it skips items, so the copy lets an observer unregister itself during a notification. Java would throw ConcurrentModificationException without it.



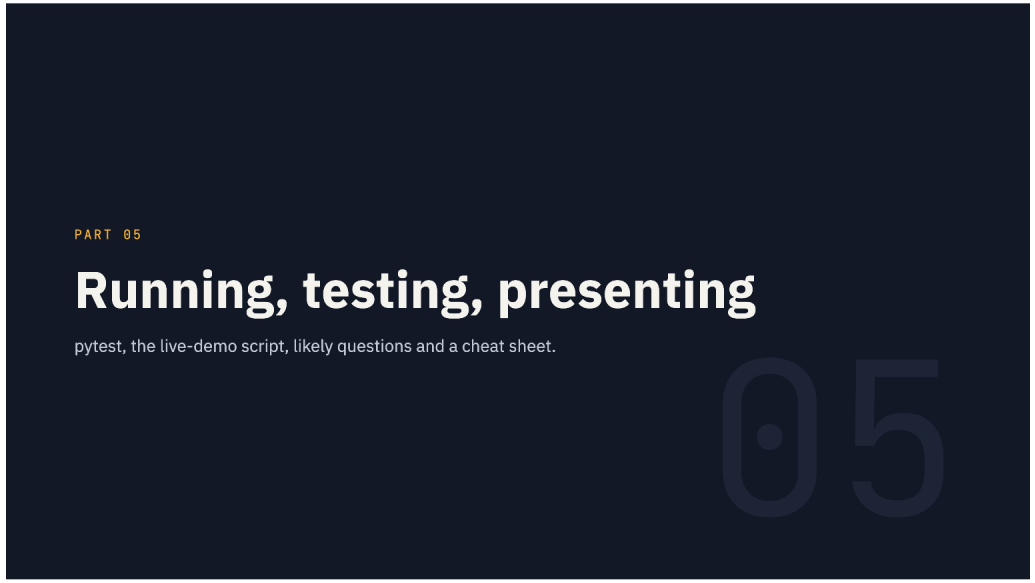
self._o.do_something without parentheses is a bound method: a method already tied to its object, which you can store and call later, like a Java method reference o::doSomething.



On the invoker slide, self.history.pop().undo() is a chained call: pop() removes and returns the last command, and undo() runs on it straight away. Java uses a Deque for the same history.



This file and Factory Method are the two places the lecture uses inheritance on purpose. Java marks the template final so subclasses can't override it; Python has typing.final, which only mypy checks.



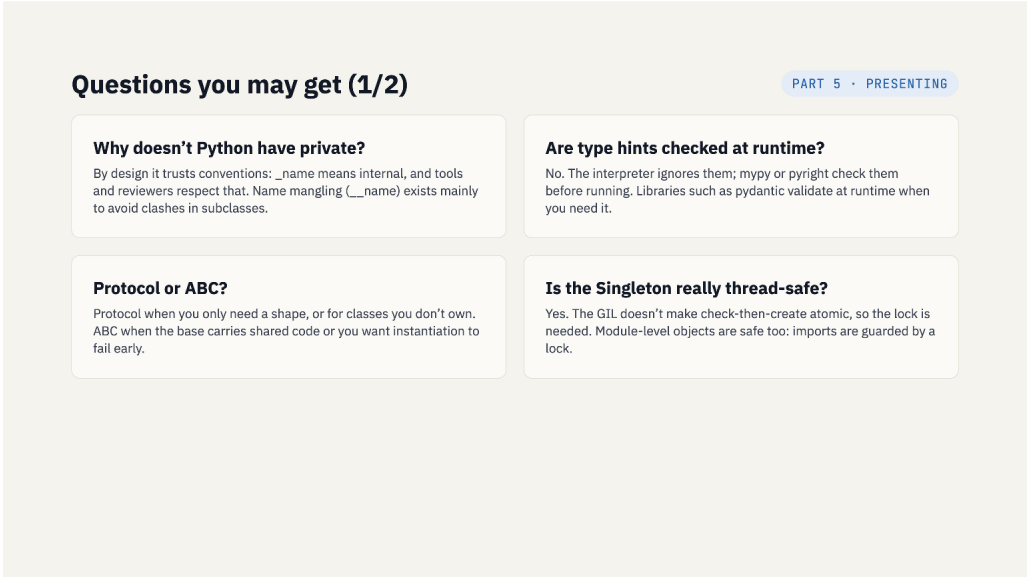
Part 5: what you need on the day.



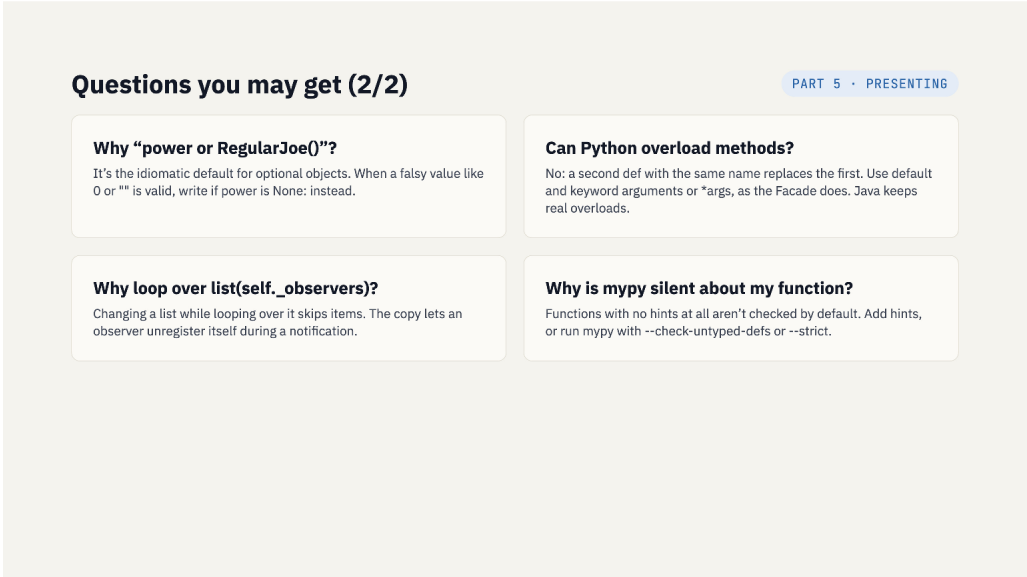
The companion folder's test_examples.py uses exactly these features, plus two built-in fixtures: capsys, which captures printed output, and tmp_path, a temporary folder. pytest passes them in by parameter name. The JUnit version on the right was compiled and run against minimal stand-ins for the JUnit annotations.



Every output in this script was checked by running it. Demo 4 is the strongest moment of the lecture: the same Liskov bug from the repo, caught by a tool before the code ever runs. To show the Java side, javac on LspAfter.java with the commented line restored gives the matching compile error.



If someone pushes on the GIL: only one thread runs Python bytecode at a time, but a thread can be paused between the None check and the assignment, so two instances are possible without the lock. Python 3.13+ also offers an optional free-threaded build without a GIL.



It’s fine to say ‘good question, let me check after the talk’. Python is not your main language, and the lecture is about design, which you know well. The Java next to every example is your safety net.

Java → Python cheat sheet

PART 5

Java	Python
this	self
null	None
true / false	True / False
&& || !	and or not
new Hero("fly")	Hero("fly")
static field	class attribute
interface	Protocol or ABC

Java	Python
class B extends A	class B(A):
@Override	nothing (or @override)
String.format("%.2f", x)	f"{x:.2f}"
for (int i=0; i<n; i++)	for i in range(n):
list.size()	len(items)
throw / catch	raise / except
try-with-resources	with … as …:

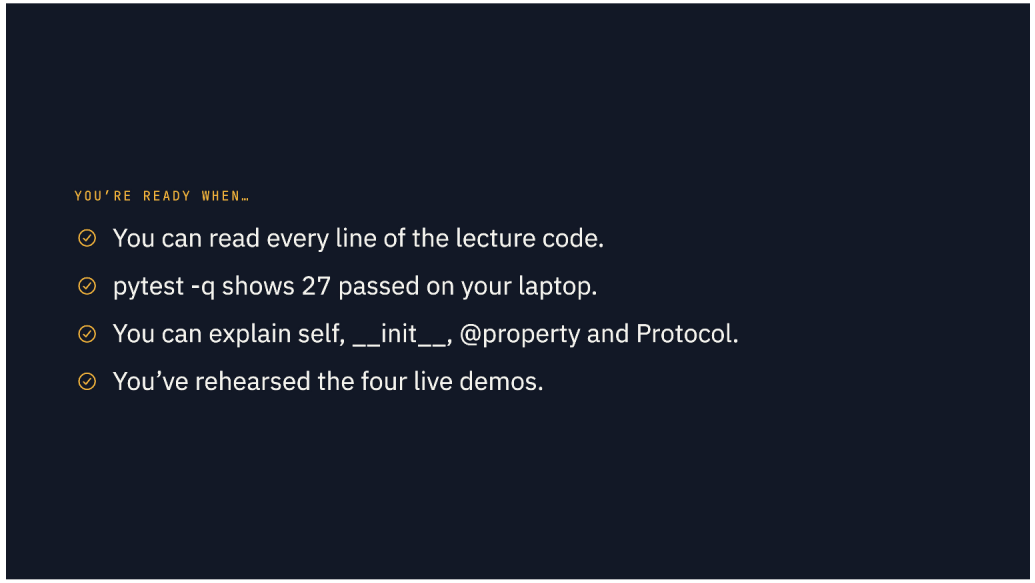
Keep this slide open on a second screen during the lecture if you can.

Lecture slide → guide slides

PART 5

Lecture slide	Syntax it uses	Guide slides
Interfaces, parts 1 and 2	ABC, @abstractmethod, Protocol	30, 31
The inheritance trap	class B(A), multiple inheritance, pass	21, 22
The same heroes, composed	*powers, a generator inside join()	12, 13
Inheritance exposes everything	subclassing list, __len__	23
S, O, L, I, D: before and after	f-strings, with open, raise, Protocol, ABC	7, 14, 15, 31
Singleton (two slides)	__new__, cls, threading.Lock, @cache	35, 40
The three factories	Enum, dataclass, Callable, a type alias	25, 26, 29, 41
Adapter, Decorator, Facade	@property, *nums, keyword-only factor	24, 39, 43
Strategy, Observer, Command	frozen dataclass, unpacking, removesuffix	38, 42, 44
Template Method (two slides)	ABC, @abstractmethod, a list over lines	45

Use this as a checklist: before the talk, open each lecture slide and the guide slides next to it.



Good luck. The design ideas are the hard part, and you already know them; Python is just the notation.